

AI -LAB MANUAL

1. Write a program to implement DFS and BFS

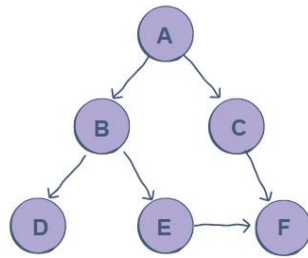
Breadth-first search (BFS) is an algorithm used for tree traversal on graphs or tree data structures. BFS can be easily implemented using recursion and data structures like dictionaries and lists.

The Algorithm

1. Pick any node, visit the adjacent unvisited vertex, mark it as visited, display it, and insert it in a queue.
2. If there are no remaining adjacent vertices left, remove the first vertex from the queue.
3. Repeat step 1 and step 2 until the queue is empty or the desired node is found.

Implementation

Consider the below graph, which is to be implemented:



SOURCE CODE:

```
graph = {  
    'A' : ['B','C'],  
    'B' : ['D', 'E'],  
    'C' : ['F'],  
    'D' : [],  
    'E' : ['F'],  
    'F' : []  
}  
  
visited = [] # List to keep track of visited nodes.  
queue = []   #Initialize a queue  
  
def bfs(visited, graph, node):
```

```

visited.append(node)
queue.append(node)

while queue:
    s = queue.pop(0)
    print (s, end = " ")

    for neighbour in graph[s]:
        if neighbour not in visited:
            visited.append(neighbour)
            queue.append(neighbour)
bfs(visited, graph, 'A')

```

OUTPUT:

A B C D E F

Explanation

- Lines 3-10: The illustrated graph is represented using an adjacency list. An easy way to do this in Python is to use a dictionary data structure, where each vertex has a stored list of its adjacent nodes.
- Line 12: `visited` is a list that is used to keep track of visited nodes.
- Line 13: `queue` is a list that is used to keep track of nodes currently in the queue.
- Line 29: The arguments of the `bfs` function are the `visited` list, the `graph` in the form of a dictionary, and the starting node `A`.
- Lines 15-26: `bfs` follows the algorithm described above:
 1. It checks and appends the starting node to the `visited` list and the `queue`.
 2. Then, while the queue contains elements, it keeps taking out nodes from the queue, appends the neighbors of that node to the queue if they are unvisited, and marks them as visited.

3.

This continues until the queue is

empty.

Time Complexity

Since all of the nodes and vertices are visited, the time complexity for BFS on a graph is $O(V + E)$; where V is the number of vertices and E is the number of edges.

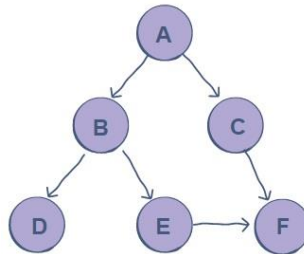
Depth-first search (DFS), is an algorithm for tree traversal on graph or tree data structures. It can be implemented easily using recursion and data structures like dictionaries and sets.

The Algorithm

1. Pick any node. If it is unvisited, mark it as visited and recur on all its adjacent nodes.
2. Repeat until all the nodes are visited, or the node to be searched is found.

Implementation

Consider the below graph, which is to be implemented:



SOURCE CODE:

Using a Python dictionary to act as an adjacency list

```
graph = {  
    'A' : ['B','C'],  
    'B' : ['D', 'E'],  
    'C' : ['F'],  
    'D' : [],  
    'E' : ['F'],  
    'F' : []  
}
```

```
visited = set() # Set to keep track of visited nodes.
```

```
def dfs(visited, graph, node):  
    if node not in visited:  
        print (node)  
        visited.add(node)  
        for neighbour in graph[node]:  
            dfs(visited, graph, neighbour)  
dfs(visited, graph, 'A')
```

OUTPUT:

A B C D E F

Explanation

- **Lines 2-9:** The illustrated graph is represented using an **adjacency list** - an easy way to do it in Python is to use a dictionary data structure. Each vertex has a list of its adjacent nodes stored.
- **Line 11:** `visited` is a set that is used to keep track of visited nodes.
- **Line 21:** The `dfs` function is called and is passed the `visited` set, the `graph` in the form of a dictionary, and `A`, which is the starting node.
- **Lines 13-18:** `dfs` follows the algorithm described above:
 1. It first checks if the current node is unvisited - if yes, it is appended in the `visited` set.
 2. Then for each neighbor of the current node, the `dfs` function is invoked again.
 3. The base case is invoked when all the nodes are visited. The function then returns.

Time Complexity

- Since all the nodes and vertices are visited, the average time complexity for DFS on a graph is $O(V + E)$, where V is the number of vertices and E is the number of edges. In case of DFS on a tree, the time complexity is $O(V)$, where V is the number of nodes.

Note: We say average time complexity because a set's in operation has an average time complexity of $O(1)$. If we used a list, the complexity would be higher.